

Reševanje ugank Nurikabe

Brin Pšunder · Optimizacijske metode

1. Uvod

Nurikabe je logična uganke na mreži. Cilj je vsako celico pobarvati kot **otok** ali **vodo** tako, da: vsako oštevilčeno polje pripada otoku s točno toliko celicami, kot pove število; otoki se med seboj ne dotikajo (po straneh); vsa voda je povezana v en kos; in nikjer ni polnega kvadrata 2×2 vode.

Program rešuje poljubne uganke dimenzije vsaj 10×10, vključno z večjimi primeri 24×14.

2. Namestitev in zagon

Potreben je Node.js (testirano z različico 20.11).

Najpreprostejši način je prek priloženega `Makefile`, ki poskrbi tudi za namestitev odvisnosti:

Namen	Ukaz
Zagon grafičnega vmesnika	<code>make run</code>
Prevajanje iz izvorne kode (izhod v <code>dist/</code>)	<code>make build</code>
Zagon testov	<code>make test</code>
Čiščenje (odstrani <code>node_modules</code> in <code>dist</code>)	<code>make clean</code>

`make run`, `make build` in `make test` najprej samodejno izvedejo `npm install`.

Brez namestitve je aplikacija dosegljiva na: <https://brinpsunder.github.io/nurikabe/> (deluje na namiznih in mobilnih napravah).

3. Zasnova in pristop

Program uganko obdelava v štirih korakih: prebere in razčleni vhod (vgrajen primer, naložena datoteka ali prilepljeno besedilo) v model mreže; **reševalnik** jo nato reši s kombinacijo **logičnega sklepanja** in **iskanja z vračanjem** (jedro programa, opisano v nadaljevanju); na koncu se rešitev izriše na platno. Reševanje teče v ločeni niti, da vmesnik ostane odziven (glej 3.7).

3.1 Model

Mreža je predstavljena z ravnimi polji, po eno vrednost na celico: **stanje** celice (neznano / voda / otok) in **pripadnost otoku** (indeks otoka ali `-1`, če je celica voda ali še neznana). Namigi so shranjeni v slovarju (položaj → ciljna velikost otoka), vsak otok pa vodi tudi množico svojih trenutnih celic. Taka predstavitev omogoča hiter dostop po indeksu in poceni razveljavljanje potez med iskanjem.

3.2 Dvoslojni pristop

Reševanje teče v dveh slojih:

- Logično sklepanje:** pravila zapolnjujejo celice, dokler kaj spreminjajo (brez ugibanja).

2. **Iskanje z vračanjem:** ko sklepanje obstane, program ugiba in se ob protislovju vrne.

Drugi sloj je nujen: ker je Nurikabe NP-poln, samo logična pravila ne zadoščajo za vse uganke.

Poenostavljena glavna zanka:

```
reši(mreža):  
    propagiraj(mreža)    # pravila do fiksne točke  
    če nastane protislovje: vrni NEUSPEH  
    če je mreža polna: vrni validiraj(mreža)  
    otok ← nedokončan otok z najmanj prostimi sosedami (MRV)  
    za vsako prosto sosednjo celico otoka:  
        poskusi celico kot otoško → rekurzivno reši  
        ob neuspehu: razveljavi potezo in jo označi kot vodo  
    vrni NEUSPEH
```

3.3 Deduktivna pravila

Pravila so razvrščena v tri stopnje po ceni izvajanja. Najprej se izvajajo poceni pravila, ki gledajo le neposredno okolico celic; ko ta obstanejo, pridejo na vrsto dražja pravila na podlagi **dosega** (kam vsak otok sploh še lahko zraste); šele ko obstanejo tudi ta, se izvedejo najdražje analize. Tako večino dela opravijo poceni pravila, draga pa se kličejo le, kadar je res potrebno.

1. stopnja: poceni pravila (neposredna okolica):

- **Dokončan otok:** otok, ki je dosegel svojo velikost, je v celoti obdan z vodo (vsaka nadaljnja širitev bi ga povečala čez dovoljeno).
- **Ločeni otoki:** celica, ki meji na dva različna otoka, mora biti voda, saj se otoka ne smeta združiti.
- **Prisilna rast:** če ima nedokončan otok eno samo prosto sosednjo celico, jo mora vzeti, sicer ne more zrasti do svoje velikosti.
- **Rast morja:** vodno območje, ki se mora še širiti in ima en sam izhod, mora ta izhod zasesti, da voda ostane povezana.
- **Zalitje morja:** ko število otoških celic doseže vsoto vseh namigov, so vsi otoki dokončani, zato so vse preostale celice voda.

2. stopnja: na podlagi dosega (ko 1. stopnja obstane):

Doseg vsakega nedokončanega otoka izračunamo z omejenim BFS: katere neznane celice sploh še lahko postanejo del tega otoka (znotraj preostale velikosti in brez vstopa v območje drugega otoka).

- **Nedosegljiva celica:** celica, ki je ne more doseči noben otok, je voda.
- **Zapolnitev otoka:** če ima otok natanko toliko dosegljivih celic, kolikor jih še potrebuje, so vse te celice otoške.

3. stopnja: drage analize (ko obstaneta tudi prejšnji):

- **Brez bazena:** če so tri celice kvadrata 2x2 že voda, mora biti četrta otoška, sicer bi nastal prepovedan bazen.
- **Nujna celica:** če bi izpustitev neke proste celice otoku onemogočila doseči svojo velikost, je ta celica nujno otoška.

3.4 Iskanje z vračanjem

Iskanje v globino izbere otok z **najmanj prostimi sosedami** (ob izenačenju tistega z najmanj manevrskega prostora), tj. po heuristiki najbolj omejene spremenljivke (MRV), in poskusi njegove širitve. Pred vejanjem **sonda** preveri vsako prosto sosednjo celico: če bi bila kot voda protislovna, je nujno otoška. Bela celica ob

izbranim otoku lahko pripada le njemu (drugače bi združila dva otoka), zato zavrnjeno ugibanje pomeni, da je ta celica voda; vsaka neuspešna veja torej pusti za sabo trajno sklepanje. Iskanje je izčrpno (za vsako prosto sosednjo celico preizkusi obe možnosti), zato najde rešitev, če obstaja; vsako dokončano mrežo pred sprejemom preveri še validacija (`validateSolution`).

3.5 Zaznavanje protislovij

Med propagacijo in iskanjem program nenehno preverja, ali je trenutno stanje sploh še rešljivo. Vejo zavrže (in se vrne) takoj, ko zazna eno od protislovij: **prevelik otok**, **preveč vode** ali otoških celic, **otok, ki ne more več doseči svoje velikosti**, prepovedan **bazen 2x2**, **razdeljeno** ali **zazidano** vodo ter **morje, ki se ne more povezati** v en kos. Zgodnje zaznavanje teh protislovij odreže velike dele iskalnega drevesa in je ključno za hitrost.

3.6 Optimizacije

Za hitrost pri velikih ugankah uporabljamo dve optimizaciji:

- **Ponovno uporabljen BFS** z generacijskimi žigi: namesto da bi pri vsakem računanju dosegla alocirali nove množice in vrste, vedno znova uporabimo ista polja in jih »počistimo« zgolj s povečanjem števca generacije.
- **Undo-dnevnik** poteze: vsaka označitev celice se zabeleži v dnevnik, zato lahko iskanje zavrnjeno vejo razveljavi z odvijanjem dnevnika, namesto da bi pri vsakem vejanju kopiralo celotno mrežo.

3.7 Ločeno izvajanje (Web Worker)

Reševanje teče v ločeni niti prek spletnega delavca (`solver.worker.ts`), zato se vmesnik med dolgimi izračuni ne zamrzne. Vmesnik in delavec se sporazumevata z izmenjavo sporočil (`postMessage`).

Vmesnik **pošlje** delavcu zahtevek:

- `solve` { `puzzleText`, `timeout`, `wantSteps` } : naročilo za reševanje.

Delavec **vrne** enega od odgovorov:

- `result` { `solved`, `elapsed`, `cells`, `islandId` } : končna rešitev in čas izračuna;
- `steps` { `solved`, `elapsed`, `snapshots` } : zaporedje posnetkov za koračni prikaz (ko je `wantSteps` resničen);
- `error` { `message` } : npr. ob neveljavnem vhodu.

Vsak nov zahtevek zažene svežega delavca in tako prekine morebitno tekoče reševanje.

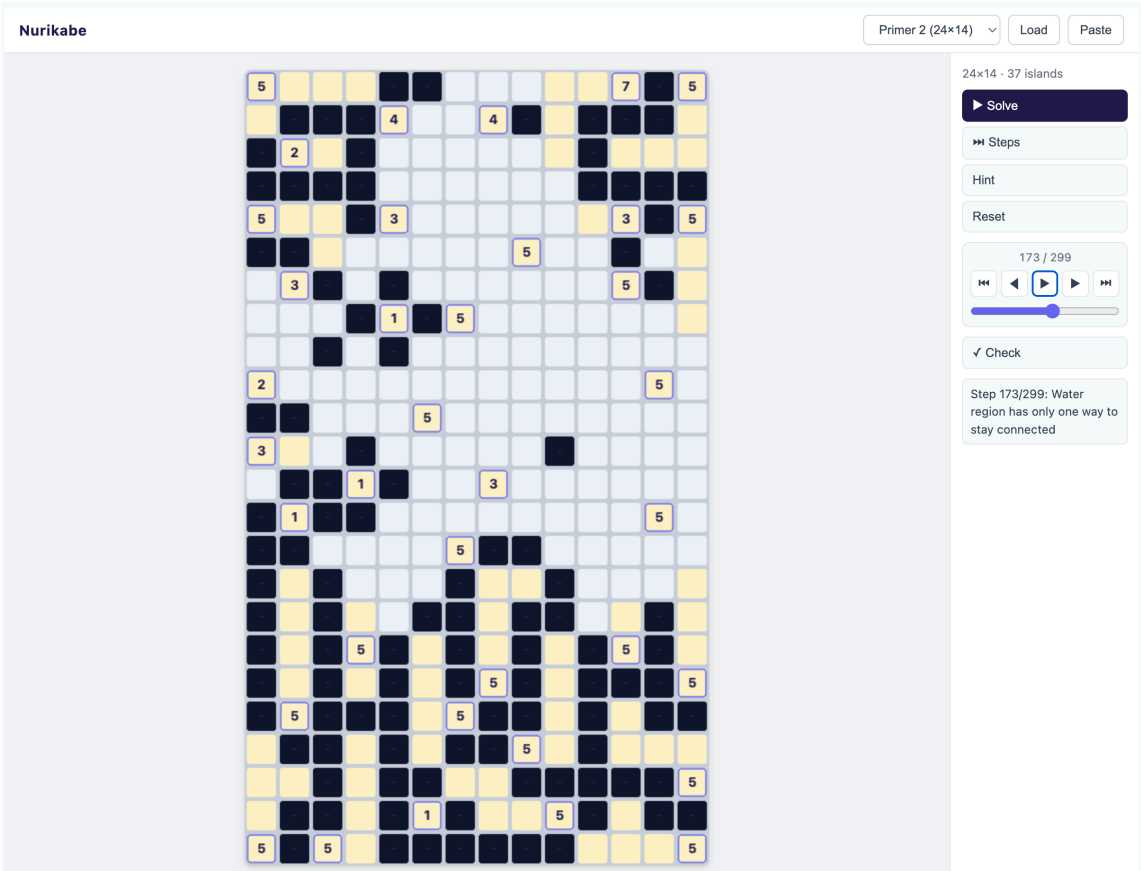
4. Grafični uporabniški vmesnik

Vmesnik je napisan v ogrodju Svelte, mrežo pa riše na platno (canvas), kar omogoča gladko izrisovanje tudi pri velikih ugankah. Podpira tri načine reševanja (avtomatsko, koračno in ročno z namigi), kar pokriva zahteve naloge, poleg tega pa sproti opozarja na napake.

- **Avtomatsko reševanje** (Solve): celotno uganko reši v ozadju. Po končanem reševanju izpiše porabljen čas in ob uspehu rešitev na kratko poudari z animacijo.
- **Koračno reševanje** (Steps): poteze se predvajajo druga za drugo, tako da je razvidno, kako se rešitev postopoma gradi. Na voljo so VCR kontrole (na začetek, korak nazaj, predvajaj/premor, korak naprej, na konec) in drsnik za hitrost predvajanja; vsak korak izpiše uporabljeno pravilo. Predvajanje lahko kadar koli ustavimo in nadaljujemo ročno z istega mesta.
- **Namig** (Hint): pokaže eno samo logično potezo in pove, katero pravilo jo utemeljuje (npr. »celica med dvema otokoma mora biti voda«), pri čemer preostanka mreže ne spremeni.

- **Ročno reševanje:** z odzivnim klikom (ali dotikom na zaslonu) celica kroži med stanji neznano → voda → otok. Tako lahko uporabnik uganko rešuje povsem sam ali pa popravi delno rešitev, dobljeno s koraki oziroma namigi.
- **Sprotno preverjanje:** med ročnim urejanjem se kršitve takoj obarvajo: oranžna označuje otok, ki je prerasel svojo velikost, rdeča pa dva otoka, ki se dotikata, ali prepovedan bazen 2x2 vode.
- **Preverjanje rešitve** (Check): preveri veljavnost celotne trenutne mreže in sporoči, ali je rešitev pravilna oziroma katero pravilo je kršeno.
- **Ponastavitev** (Reset): povrne uganko v začetno stanje (samo z namigi), da lahko reševanje začnemo znova.
- **Nalaganje uganke:** vgrajeni primeri prek spustnega menija, naložena datoteka `.txt` ali besedilo, prilepljeno v pogovorno okno.

Pogoste ukaze je mogoče sprožiti tudi s tipkovnico (npr. preslednica za samodejno reševanje, puščici za premikanje med koraki).



Slika 1: Koračno reševanje primera 2; vidni so trenutni korak (173/299), uporabljeno pravilo in kontrole predvajanja.

5. Rešeni primeri in časi

Spodnja tabela prikazuje čase reševanja (izmerjene v brskalniku) in število vozlišč iskanja (pridobljeno z `make test`) za priložene primere.

Primer	Dimenzije	Št. namigov	Čas	Vozlišča iskanja
easy	5×5	2	0.004 s	4
medium	10×10	18	0.006 s	1
hard	20×20	70	0.048 s	21
primer1	18×10	24	0.018 s	7
primer2	24×14	37	0.030 s	8
primer3	24×14	38	1.082 s	5480
vrazji1	24×14	39	0.358 s	1530
vrazji2	24×14	40	0.961 s	3227

Nurikabe

Primer 1 (18×10)LoadPaste

4

6

3

5

4

1

3

5

2

2

5

6

4

1

6

4

1

1

5

3

2

3

5

1

18×10 · 24 islands

Solve

Steps

Hint

Reset

Check

0.018s

Solved in 0.018s

Slika 2: Rešen primer 1 (18×10).

Nurikabe

Primer 2 (24×14)
Load
Paste

5													7		5
				4			4								
	2														
5				3								3		5	
							5								
	3											5			
				1		5									
2														5	
				5											
3															
			1				3								
	1													5	
						5									
			5									5			
							5							5	
	5					5									
							5								
															5
						1			5						
5		5													5

24×14 · 37 islands

Solve
Steps
Hint
Reset
Check

0.027s

Solved in 0.027s

Slika 3: Rešen primer 2 (24×14).

Nurikabe

Primer 3 (24×14)
Load
Paste

1

2

1

2

1

2

3

2

4

1

18

2

2

4

3

2

4

1

18

4

2

1

2

3

6

3

8

1

2

2

5

1

3

6

6

4

7

2

5

14

24×14 · 38 islands

Solve

Steps

Hint

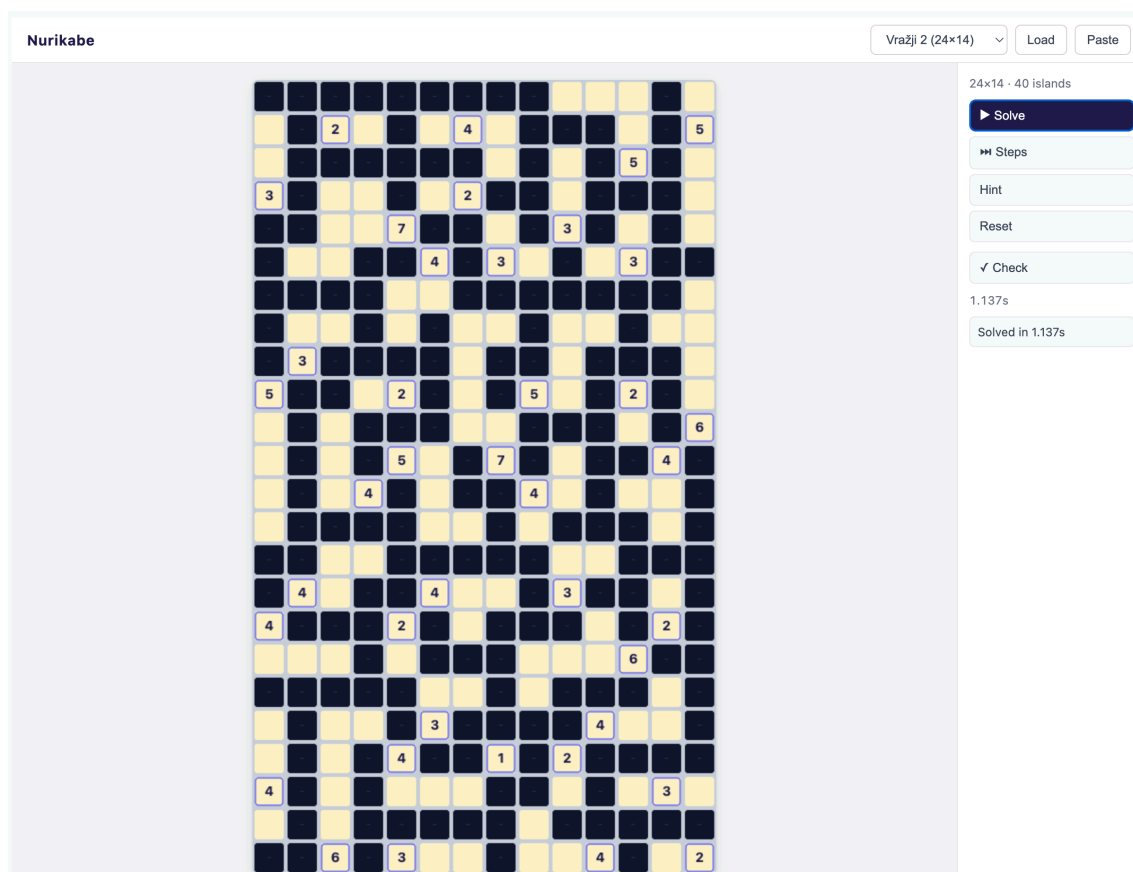
Reset

Check

1.094s

Solved in 1.094s

Slika 4: Rešen primer 3 (24×14); status prikazuje čas reševanja.



Slika 5: Rešen primer vražji 2 (24x14).

6. Struktura projekta in testiranje

```
src/
  main.ts          vstopna točka
  App.svelte       GUI (platno, kontrole, ročno reševanje)
  lib/
    puzzle.ts      model mreže, branje vhoda, validacija
    solver.ts      sklepanje + iskanje z vračanjem
    solver.worker.ts ovoj za Web Worker
  tests/           testi (puzzles, solver, helpers)
  public/puzzles/  vhodne datoteke
```

Vhodne datoteke (`public/puzzles/`): števila ločena s presledki, `0` ali `.` pomeni prazno polje, pozitivno število je namig. Priloženih je 8 primerov: `easy_5x5` , `medium_10x10` , `hard_20x20` , `primer1_18x10` , `primer2_24x14` , `primer3_24x14` , `vrazji1_24x14` , `vrazji2_24x14` .

Testiranje (TDD)

Reševalnik je nastal po pristopu **TDD** (test-driven development): testi pravil so bili napisani pred implementacijo. Pravilnost preverja 49 testov (`make test` , ogrodje vitest), razvrščenih v tri skupine:

- **enotski testi reševalnika:** vsako od 9 deduktivnih pravil in zaznavanje protislovij imata pozitivni in negativni primer, poleg tega so pokrite še pomožne funkcije (izračun ciljev, sinhronizacija otokov), propagacija, namig in robna primera, da se nerešljiva uganka varno konča (ne obvisi) in da se upošteva časovna omejitev, preverjena z vbrizgano lažno uro (kar preizkus naredi determinističen);
- **integracijski preizkus:** vseh 8 priloženih ugank se mora rešiti, prestati validacijo in ostati znotraj časovne omejitve (test izpiše tudi čas in število vozlišč iskanja);
- **osnovni preizkusi razčlenjevalnika in pomožnih funkcij:** branje vhoda in gradnja mreže iz opisa.

Repozitorij: <https://github.com/brinpsunder/nurikabe>

7. Viri

Vsi navedeni viri so prosto dostopni; pri vsakem je navedeno, katere ideje smo iz njega uporabili.

- **Nurikabe — Wikipedia:** pravila uganke, validacija rešitve in podatek o NP-polnosti.
[https://en.wikipedia.org/wiki/Nurikabe_\(puzzle\)](https://en.wikipedia.org/wiki/Nurikabe_(puzzle))
- **Conceptis Puzzles — Nurikabe solving techniques:** človeške tehnike sklepanja, ki so osnova deduktivnih pravil (dosegljivost, prisilna rast, zapolnitev otoka, nujna celica, rast morja), ter pogled naprej s predpostavkami (»looking ahead«), na katerem temelji sonda.
<https://www.conceptispuzzles.com/index.aspx?uri=puzzle/nurikabe/techniques>
- **Russell, Norvig — Artificial Intelligence: A Modern Approach, poglavje Constraint Satisfaction Problems** (prosto dostopno na uradni strani avtorjev): iskanje z vračanjem, propagacija omejitev in heuristika najbolj omejene spremenljivke (MRV). <https://aima.cs.berkeley.edu/newchap05.pdf>